Université de Cergy-Pontoise

Introduction to Data Sciences – TD Classification and Clustering

Part I Examples

Write the comments to explain how the following programs work.

A) Classification

Example 1

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree

balance_data = pd.read_csv('balance-scale.data', sep= ',', header= None)
balance_data.shape
X = balance_data.values[:, 1:5]
Y = balance_data.values[:,0]
X_train, X_test, y_train, y_test = train_test_split( X, Y, test_size = 0.3)

clf_entropy = DecisionTreeClassifier(criterion = "entropy", max_depth=3,
min_samples_leaf=5)
clf_entropy.fit(X_train, y_train)
y_pred_en = clf_entropy.predict(X_test)
print ("Accuracy is ", accuracy_score(y_test,y_pred_en)*100)
```

Example 2

```
import numpy as np
...
import graphviz

Jogging_data=pd.read_csv('JoggingTitre.csv', sep=',')
y=Jogging_data['Jogging']
x=Jogging_data.drop(['Jogging'], axis=1)
x_dum=pd.get_dummies(x)
x_dum
# partiel dummies : sub_dum=pd.get_dummies(x, columns=['Temps', 'Vent'])

clf_entropy = DecisionTreeClassifier(criterion = "entropy")
outputTree=clf_entropy.fit(x_dum, y)

dot_data = tree.export_graphviz(outputTree, out_file=None)
graph = graphviz.Source(dot_data)
graph.render("Td2_dum01")

dot_data = tree.export_graphviz(outputTree, out_file=None, feature_names =
x_dum.columns)
graph = graphviz.Source(dot_data)
graph.render("Td2_dum01Name")
#Check and compare the files Td2_dum01.pdf and Td2_dum01Name.pdf
```

Example 3

```
import numpy as np
...
from sklearn.model_selection import KFold

balance_data = pd.read_csv('balance-scale.data', sep= ',', header= None)
X = balance_data.values[:, 1:5]
Y = balance_data.values[:,0]

kfold = KFold(10, True, 10)
ac=0.0
ac_score=0.0
for train, test in kfold.split(X):
        X_train, X_test, y_train, y_test = X[train], X[test], Y[train], Y[test]
        clf_entropy.fit(X_train, y_train)
        y_pred_en = clf_entropy.predict(X_test)
        ac_score=accuracy_score(y_test,y_pred_en)*100
        ac=ac+ac_score
        print ("Accuracy is ", ac_score)

ac_avg=ac/10
print ("Average Accuracy is ", ac_avg)
```

B) Clustering

Example 4

```
import numpy as np

X = np.array([[1], [2], [3], [6], [7], [8], [13],[15], [17]])

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, init=np.array([[1], [2], [3]]))
kmeans.fit(X)
kmeans.cluster_centers_
kmeans.labels_

from sklearn.cluster import AgglomerativeClustering
cluster = AgglomerativeClustering(n_clusters=3, linkage='single')
cluster.fit(X)
cluster.labels_
```

Example 5

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import dendrogram, linkage
Y = np.array([[1], [2], [4], [7], [8], [10], [15],[17], [21]])
linked = linkage(Y, 'single')
labelList = [[1], [2], [4], [7], [8], [10], [15],[17], [21]]

plt.figure(figsize=(10, 7))
```

```
dendrogram(linked,
        orientation='top',
        labels=labelList,
        distance_sort='descending',
        show_leaf_counts=True)
plt.show()
```

Example 6

```
import numpy as np
Z = np.array([[11], [21], [22], [23], [26], [27], [28], [33],[35], [37], [47]])

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, init=np.array([[1], [2], [3]]))
kmeans.fit(Z)
kmeans.cluster_centers_
kmeans.labels_

from sklearn.cluster import AgglomerativeClustering
cluster = AgglomerativeClustering(n_clusters=3, linkage='single')
cluster.fit(Z)
cluster.labels_
```

Example 7

```
import pandas as pd
import numpy as np
import sklearn.metrics as sm
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn import datasets
from sklearn import metrics

iris = datasets.load_iris()
print(iris)
print(iris.data)
print(iris.feature_names)
print(iris.target)
print(iris.target_names)

x=pd.DataFrame(iris.data)
x.columns=['Sepal_Length','Sepal_width','Petal_Length','Petal_width']
y=pd.DataFrame(iris.target)
y.columns=['Targets']
model=KMeans(n_clusters=3)
model.fit(x)

colormap=np.array(['r','g','b'])
plt.scatter(x.Petal_Length, x.Petal_width,c=colormap[y.Targets],s=40)
plt.show()

plt.scatter(x.Petal_Length, x.Petal_width,c=colormap[model.labels_],s=40)
plt.show()
metrics.adjusted_rand_score(model.labels_, y.Targets)
```

Part II Exercises

Execute the following analyses and observations for ***Exercise1*** and ***Exercise2*** with two data sets :

car.data and ***tic-tac-toe.data.***

There are some information for these data sets in the ****.name*** files. You can get more of information related to these data sets from the UCI Web site

https://archive.ics.uci.edu/ml/datasets.html

Exercise1

- When using the ***DecisionTreeClassifier***, we can specify the parameter ***max_depth*** to limit the maximal depth of the decision tree. Try the *max_depth* from 3 to 10 (or to the real maximal depth of the classifier) to observe whether the maximal depth brings some impacts to the accuracy of the classifier.
- For each *max_depth*, you use the K-fold cross-validation method, for K=10, to compute the average accuracy of the classifier.
- Are there relationships among the size of data sets (the number of attributes and the number of records etc.), Decision Tree Depth, and the accuracy of classifier?
- Write your observations and put some graphical representations for a better understanding your description.

Exercise2

- Try to build the decision Tree Classifier with different sizes of training data set (10%, 25%, 33%, 50%, 66% and 75% of the whole data set).
- For each size of training data set, use the **R*andom Sub-sampling*** method to computer the average accuracy of the classifier. More precisely, you extract the randomly the same size of training data set to train the classifier and test its accuracy with the other part of data set. Repeat 10 times this procedure to compute the average accuracy.
- Does it exist relationships between the size of training data and the classifier accuracy? With some graphical representations, explain your observations.

Exercise 3

Generate and explain with help of the ***dendrogram*** how the four linkages methods (single, average, and complete) work in the Agglomerative Clustering method.

Exercise 4

Comparing the data sets Y and Z in Example 5 and 6 and the result of these examples, what kinds of observation can you get from the two different clustering methods.

Exercise 5

In Example 7, a clustering method is used to classify data. The *metrics.adjusted_rand_score* can be used to evaluate the performance of such classifier. Compare the performances of the classifiers from the *Agglomerative Clustering* method with different linkage method (single, average, complete, and ware) as well as the performance of the *K-means* classifier.

Final Report

In your report, you have to return

- the programs with comments of Part I;
- python programs with comments for the exercises in Part II
- observations/comparisons from Exercise 1, 2, 4 and 5 of Part II